



Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» ім. Ігоря Сікорського

Лабораторна робота №1
з дисципліни «Введення до операційних систем»

«Планування процесів»

Перевірив:
Сергієнко П. А.
Виконав:
студент групи КВ-32
Доній Олександр
Варіант: 4

Київ 2026

Загальне завдання

1. Написати програму, що моделює послідовність обслуговування процесів, які знаходяться у черзі готових до виконання, за певним алгоритмом планування (перелік варіантів представлений нижче).

Вхідні дані (студент задає самостійно з урахуванням особливостей заданого алгоритму):

- послідовність процесів, які надходять до черги готових процесів;
- час надходження процесів до черги;
- очікуваний час виконання;
- пріоритет процесів та/або інші додаткові дані, що необхідні для реалізації алгоритму планування згідно завдання.

Програма повинна обчислювати:

- час початку виконання для кожного процесу;
- час завершення для кожного процесу;
- час затримки (очікування) в черзі для кожного процесу;
- повний час виконання (з урахуванням затримки) для кожного процесу;
- середній час очікування процесів у черзі;
- середній повний час виконання процесів (з урахуванням затримки);
- інші результати, необхідні для оцінки алгоритму планування.

2. Зробити візуалізацію роботи програми у часі (на різних наборах вхідних даних, що ілюструють особливості алгоритму), а також представити кінцеві результати роботи на екрані у вигляді таблиці (як, наприклад, у табл. 1.5), діаграми, графіку, малюнку тощо

3. Проаналізувати та пояснити отримані результати. Надати висновки щодо алгоритму планування за результатами роботи.

Варіант

Розробити окремі програми для кожного з нижчезазначених алгоритмів та порівняти результати їх роботи:

1) алгоритм FCFS

2) алгоритм RR

Лістинг програми мовою C++

FCFS:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>

using namespace std;

struct Process {
    int id;
    int arrival_time; // t1
    int burst_time;   // t2
    int start_time;   // t3
    int completion_time; // t4
    int waiting_time; // t5
    int turnaround_time; // t6
};

// Функція для сортування за часом надходження
bool compareArrival(Process p1, Process p2) {
    return p1.arrival_time < p2.arrival_time;
}

// Функція для сортування за ID (для фінального виводу)
bool compareID(Process p1, Process p2) {
    return p1.id < p2.id;
}

int main() {
    int n;
    cout << "---- Алгоритм FCFS ----\n";
    cout << "Введіть кількість процесів: ";
    cin >> n;

    vector<Process> p(n);
    for (int i = 0; i < n; i++) {
```

```

    p[i].id = i + 1;
    cout << "Процес " << p[i].id << " - Час надходження (t1): ";
    cin >> p[i].arrival_time;
    cout << "Процес " << p[i].id << " - Очікуваний час виконання (t2): ";
    cin >> p[i].burst_time;
}

// Сортуємо процеси за часом надходження
sort(p.begin(), p.end(), compareArrival);

int current_time = 0;
float total_waiting_time = 0;
float total_turnaround_time = 0;

vector<pair<int, int>> timeline; // Для візуалізації: {ID процесу, час виконання}

for (int i = 0; i < n; i++) {
    // Якщо процесор простояє до прибуття наступного процесу
    if (current_time < p[i].arrival_time) {
        timeline.push_back({0, p[i].arrival_time - current_time}); // 0 означає
простій
        current_time = p[i].arrival_time;
    }

    p[i].start_time = current_time;
    p[i].completion_time = current_time + p[i].burst_time;
    p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
    p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;

    total_waiting_time += p[i].waiting_time;
    total_turnaround_time += p[i].turnaround_time;

    timeline.push_back({p[i].id, p[i].burst_time});
    current_time = p[i].completion_time;
}

// Повертаємо оригінальний порядок для виводу таблиці
sort(p.begin(), p.end(), compareID);

cout << "\nТаблиця результатів:\n";
cout << "-----\n";
cout << "-----\n";
cout << "| No | Час надх. (t1) | Очік. час (t2) | Початок (t3) | Завершення (t4) |  
Очікування (t5) | Повний (t6) |\n";
cout << "-----\n";
cout << "-----\n";

```

```

for (int i = 0; i < n; i++) {
    cout << "|" << setw(2) << p[i].id
        << " |" << setw(14) << p[i].arrival_time
        << " |" << setw(14) << p[i].burst_time
        << " |" << setw(12) << p[i].start_time
        << " |" << setw(15) << p[i].completion_time
        << " |" << setw(15) << p[i].waiting_time
        << " |" << setw(11) << p[i].turnaround_time << " |\n";
}
cout << "-----\n";
-----\n";

cout << "\nСередній час очікування: " << total_waiting_time / n;
cout << "\nСередній повний час виконання: " << total_turnaround_time / n << "\n";

cout << "\nВізуалізація виконання (Діаграма Ганта):\n";
int time_counter = timeline[0].first == 0 ? 0 : p[0].start_time;
cout << time_counter;
for (size_t i = 0; i < timeline.size(); i++) {
    if (timeline[i].first == 0) cout << " [Простий] ";
    else cout << " [P" << timeline[i].first << " ] ";
    time_counter += timeline[i].second;
    cout << time_counter;
}
cout << "\n";

return 0;
}

```

RR:

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <queue>

using namespace std;

struct Process {
    int id;
    int arrival_time; // t1
    int burst_time;   // t2
    int start_time;   // t3
    int completion_time; // t4
    int waiting_time; // t5
}

```

```

        int turnaround_time; // t6
        int remaining_time;
        bool is_started;
};

bool compareArrival(Process p1, Process p2) {
    return p1.arrival_time < p2.arrival_time;
}

bool compareID(Process p1, Process p2) {
    return p1.id < p2.id;
}

int main() {
    int n, quantum;
    cout << "---- Алгоритм RR (Round Robin) ----\n";
    cout << "Введіть кількість процесів: ";
    cin >> n;
    cout << "Введіть квант часу: ";
    cin >> quantum;

    vector<Process> p(n);
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        cout << "Процес " << p[i].id << " - Час надходження (t1): ";
        cin >> p[i].arrival_time;
        cout << "Процес " << p[i].id << " - Очікуваний час виконання (t2): ";
        cin >> p[i].burst_time;
        p[i].remaining_time = p[i].burst_time;
        p[i].is_started = false;
    }

    sort(p.begin(), p.end(), compareArrival);

    queue<int> ready_queue;
    vector<bool> in_queue(n, false);
    vector<pair<int, int>> timeline;

    int current_time = 0;
    int completed = 0;
    int idx = 0; // Індекс процесу для перевірки прибуття

    // Додаємо процеси, що прибули в момент часу 0
    while (idx < n && p[idx].arrival_time <= current_time) {
        ready_queue.push(idx);
        in_queue[idx] = true;
    }

```

```

        idx++;
    }

    while (completed != n) {
        if (ready_queue.empty()) {
            // Якщо черга порожня, процесор простояє до наступного процесу
            int next_arrival = p[idx].arrival_time;
            timeline.push_back({0, next_arrival - current_time});
            current_time = next_arrival;

            while (idx < n && p[idx].arrival_time <= current_time) {
                ready_queue.push(idx);
                in_queue[idx] = true;
                idx++;
            }
        }

        int current_process = ready_queue.front();
        ready_queue.pop();
        in_queue[current_process] = false;

        // Фіксуємо час першого запуску
        if (!p[current_process].is_started) {
            p[current_process].start_time = current_time;
            p[current_process].is_started = true;
        }

        int execute_time = min(p[current_process].remaining_time, quantum);
        p[current_process].remaining_time -= execute_time;
        current_time += execute_time;

        timeline.push_back({p[current_process].id, execute_time});

        // Перевіряємо, чи прибули нові процеси під час виконання поточного
        while (idx < n && p[idx].arrival_time <= current_time) {
            ready_queue.push(idx);
            in_queue[idx] = true;
            idx++;
        }

        // Якщо процес ще не завершився, повертаємо його в кінець черги
        if (p[current_process].remaining_time > 0) {
            ready_queue.push(current_process);
            in_queue[current_process] = true;
        } else {
            // Процес завершено

```

```

        p[current_process].completion_time = current_time;
        p[current_process].turnaround_time = p[current_process].completion_time -
p[current_process].arrival_time;
        p[current_process].waiting_time = p[current_process].turnaround_time -
p[current_process].burst_time;
        completed++;
    }
}

float total_waiting_time = 0;
float total_turnaround_time = 0;
for (int i = 0; i < n; i++) {
    total_waiting_time += p[i].waiting_time;
    total_turnaround_time += p[i].turnaround_time;
}

sort(p.begin(), p.end(), compareID);

cout << "\nТаблиця результатів:\n";
cout << "-----\n";
cout << "| No | Час надх. (t1) | Очік. час (t2) | Початок (t3) | Завершення (t4) |
Очікування (t5) | Повний (t6) |\n";
cout << "-----\n";
for (int i = 0; i < n; i++) {
    cout << "| " << setw(2) << p[i].id
        << " | " << setw(14) << p[i].arrival_time
        << " | " << setw(14) << p[i].burst_time
        << " | " << setw(12) << p[i].start_time
        << " | " << setw(15) << p[i].completion_time
        << " | " << setw(15) << p[i].waiting_time
        << " | " << setw(11) << p[i].turnaround_time << " |\n";
}
cout << "-----\n";

cout << "\nСередній час очікування: " << total_waiting_time / n;
cout << "\nСередній повний час виконання: " << total_turnaround_time / n << "\n";

cout << "\nВізуалізація виконання (Діаграма Ганта):\n";
int time_counter = 0;
cout << time_counter;
for (size_t i = 0; i < timeline.size(); i++) {
    if (timeline[i].first == 0) cout << " [Простій] ";
    else cout << " [Р" << timeline[i].first << " ] ";
}

```



```

        time_counter += timeline[i].second;
        cout << time_counter;
    }
    cout << "\n";

    return 0;
}

```

Тестування

```

--- Алгоритм FCFS ---
Введіть кількість процесів: 5
Процес 1 - Час надходження (t1): 0
Процес 1 - Очікуваний час виконання (t2): 10
Процес 2 - Час надходження (t1): 1
Процес 2 - Очікуваний час виконання (t2): 2
Процес 3 - Час надходження (t1): 2
Процес 3 - Очікуваний час виконання (t2): 4
Процес 4 - Час надходження (t1): 3
Процес 4 - Очікуваний час виконання (t2): 1
Процес 5 - Час надходження (t1): 4
Процес 5 - Очікуваний час виконання (t2): 5

Таблиця результатів:
-----
| No | Час надх. (t1) | Очік. час (t2) | Початок (t3) | Завершення (t4) | Очікування (t5) | Повний (t6) |
-----
| 1 | 0 | 10 | 0 | 10 | 0 | 10 |
| 2 | 1 | 2 | 10 | 12 | 9 | 11 |
| 3 | 2 | 4 | 12 | 16 | 10 | 14 |
| 4 | 3 | 1 | 16 | 17 | 13 | 14 |
| 5 | 4 | 5 | 17 | 22 | 13 | 18 |
-----

Середній час очікування: 9
Середній повний час виконання: 13.4

Візуалізація виконання (Діаграма Ганта):
0 [ P1 ] 10 [ P2 ] 12 [ P3 ] 16 [ P4 ] 17 [ P5 ] 22

```

```

--- Алгоритм RR (Round Robin) ---
Введіть кількість процесів: 5
Введіть квант часу: 3
Процес 1 - Час надходження (t1): 0
Процес 1 - Очікуваний час виконання (t2): 10
Процес 2 - Час надходження (t1): 1
Процес 2 - Очікуваний час виконання (t2): 2
Процес 3 - Час надходження (t1): 2
Процес 3 - Очікуваний час виконання (t2): 4
Процес 4 - Час надходження (t1): 3
Процес 4 - Очікуваний час виконання (t2): 1
Процес 5 - Час надходження (t1): 4
Процес 5 - Очікуваний час виконання (t2): 5

Таблиця результатів:
-----
| No | Час надх. (t1) | Очік. час (t2) | Початок (t3) | Завершення (t4) | Очікування (t5) | Повний (t6) |
-----
| 1 | 0 | 10 | 0 | 22 | 12 | 22 |
| 2 | 1 | 2 | 3 | 5 | 2 | 4 |
| 3 | 2 | 4 | 5 | 16 | 10 | 14 |
| 4 | 3 | 1 | 8 | 9 | 5 | 6 |
| 5 | 4 | 5 | 12 | 21 | 12 | 17 |
-----

Середній час очікування: 8.2
Середній повний час виконання: 12.6

Візуалізація виконання (Діаграма Ганта):
0 [ P1 ] 3 [ P2 ] 5 [ P3 ] 8 [ P4 ] 9 [ P1 ] 12 [ P5 ] 15 [ P3 ] 16 [ P1 ] 19 [ P5 ] 21 [ P1 ] 22

```

В ході виконання лабораторної роботи було змодельовано роботу двох алгоритмів планування процесів: FCFS (First Come First Serve) та RR (Round Robin). Для тестування було використано набір з 5 процесів, де перший процес

(P1) був найдовшим (очікуваний час виконання $t_2 = 10$), а інші — значно коротшими.

1. Результати алгоритму FCFS: Алгоритм FCFS є невитісняльною стратегією, тобто процеси виконуються у порядку їх надходження до системи. Аналіз таблиці показує виникнення так званого "ефекту конвою":

- Процес P1, який надійшов першим ($t_1 = 0$), монополізував процесор на весь час свого виконання (до $t_4 = 10$).

- Через це короткі процеси P2 ($t_2 = 2$) та P4 ($t_2 = 1$), які надійшли майже одразу після P1, були змушені довго очікувати в черзі. Наприклад, час очікування (t_5) для короткого P4 склав аж 13 одиниць часу.

- Як наслідок, середній час очікування для всіх процесів склав 9 одиниць, а середній повний час виконання — 13.4.

2. Результати алгоритму RR: Алгоритм RR працює за принципом кругового планування, де кожен процес отримує фіксований квант часу. У нашому тесті квант дорівнював 3 одиницям.

- На діаграмі Ганта чітко видно, що довгий процес P1 переривався, поступаючись місцем іншим процесам.

- Це кардинально змінило ситуацію для коротких задач. Процес P2 завершився вже на 5-й одиниці часу (замість 12-ї у FCFS), а процес P4 — на 9-й (замість 17-ї). Час їхнього очікування (t_5) склав всього 2 та 5 одиниць відповідно.

- Хоча повний час виконання довгого P1 збільшився (з 10 до 22), загальна ефективність системи зросла. Середній час очікування зменшився до 8.2, а середній повний час виконання — до 12.6.

Висновок

Під час виконання лабораторної роботи було досліджено два алгоритми планування: безпріоритетний невитісняльний (FCFS) та витісняльний круговий (RR).

1. **Алгоритм FCFS** продемонстрував свою простоту, але виявив суттєвий недолік при роботі з процесами різної тривалості. Якщо до системи першим потрапляє процес, що вимагає значного процесорного часу, він блокує виконання всіх наступних, навіть дуже коротких задач. Це призводить до різкого збільшення середнього часу очікування.
2. **Алгоритм RR** довів свою ефективність у вирішенні проблеми монополізації процесора. Завдяки поділу часу на кванти, система своєчасно реагує на всі процеси в черзі. Короткі завдання завершуються набагато швидше, що мінімізує час відгуку, тому цей алгоритм є кращим для інтерактивних процесів.

Порівняння результатів на ідентичних вхідних даних показує, що для багатозадачних систем алгоритм Round Robin є більш справедливим і гнучким, оскільки він дозволяє зменшити середній час очікування процесів у черзі порівняно з алгоритмом FCFS.